Module 1

Introduction

- What is ChatGPT?
- Released in November 2022
- Built using Large Language Models (LLMs)
- One of the fastest-growing AI applications
- 100 Million Monthly Users (January 2023)
- Fastest-growing consumer application

Application	Time to Reach 100M Users
ChatGPT	2 Months
TikTok	9 Months
Instagram	2.5 Years

Traditional Search Engine

- Search keywords
- Read multiple websites
- Compare information
- Find final answer

ChatGPT

- Ask in natural language
- Get direct answer
- Ask follow-up questions
- Personalized explanation

- Students can use ChatGPT to
- Learn new concepts
- Get personalized tutorials
- Ask unlimited questions
- Practice coding
- Summarize notes

- ChatGPT helps programmers by
- Writing code
- Explaining code
- Debugging errors
- Creating projects
- Auto-completing code

- Future AI Assistants may
- Talk naturally
- Understand voice
- Work across applications
- Perform tasks automatically
- Become everyday companions

AI will become a digital personal assistant.

Everyday Applications

LLMs can help in

- Customer Support
- Banking
- Healthcare
- Education
- Travel
- Shopping
- Smart Homes

Personalized AI Assistant

- A Personal AI Assistant can
- Morning
- Read news
- Afternoon
- Schedule meetings
- Evening
- Book travel
- Night
- Help with shopping

Benefits of LLMs

- Saves time
- Increases productivity
- Available anytime
- Personalized learning
- Easy communication
- Helps solve problems

Limitations of ChatGPT

- Can give incorrect answers
- Depends on prompt quality
- May generate outdated information
- Cannot replace human thinking
- Privacy concerns

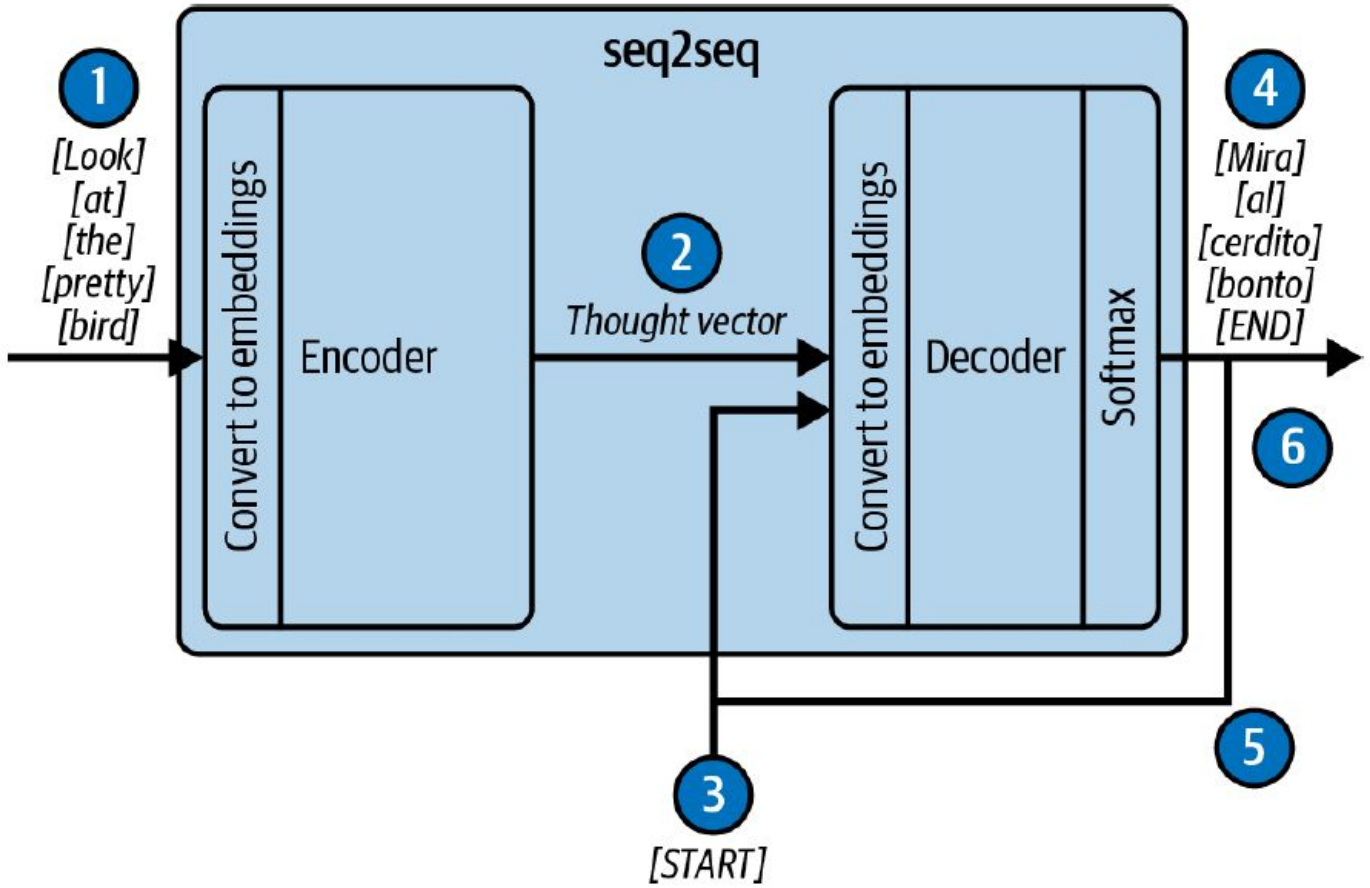
What is language model?

Seq2seq

- Sequence to sequence (seq2seq) architecture introduced at Google.
- Seq2seq was a recurrent neural network, - text processing because it processes one token at a time and recurrently updates its internal state.
- Process long sequences of text.
- Performing several different types of natural language tasks: classification, entity extraction, translation, summarization, and more.
- But these models had an Achilles' heel—an information bottleneck limited their capabilities.

- 2 major components: the encoder and the decoder
- Send stream of tokens to encoder (process one at a time)
- Encoder updates the hidden state vector that accumulates information
- Final value (thought vector)
- Decoder – generate output tokens
- Problem : thought vector is fixed and finate
- “forgets” important information from longer blocks - information bottleneck

Seq2Seq Architecture



Working of the model

1. Tokens from the source language are sent to the encoder one at a time and converted to an embedding vector, and they update the internal state of the encoder.

2. The internal state is packaged up as the thought vector and sent to the decoder.

3. A special “start” token is sent to the decoder, indicating that this is the start of the output tokens.

4. Conditioned upon the value of the thought vector, the decoder state is updated and an output token from the target language is emitted.

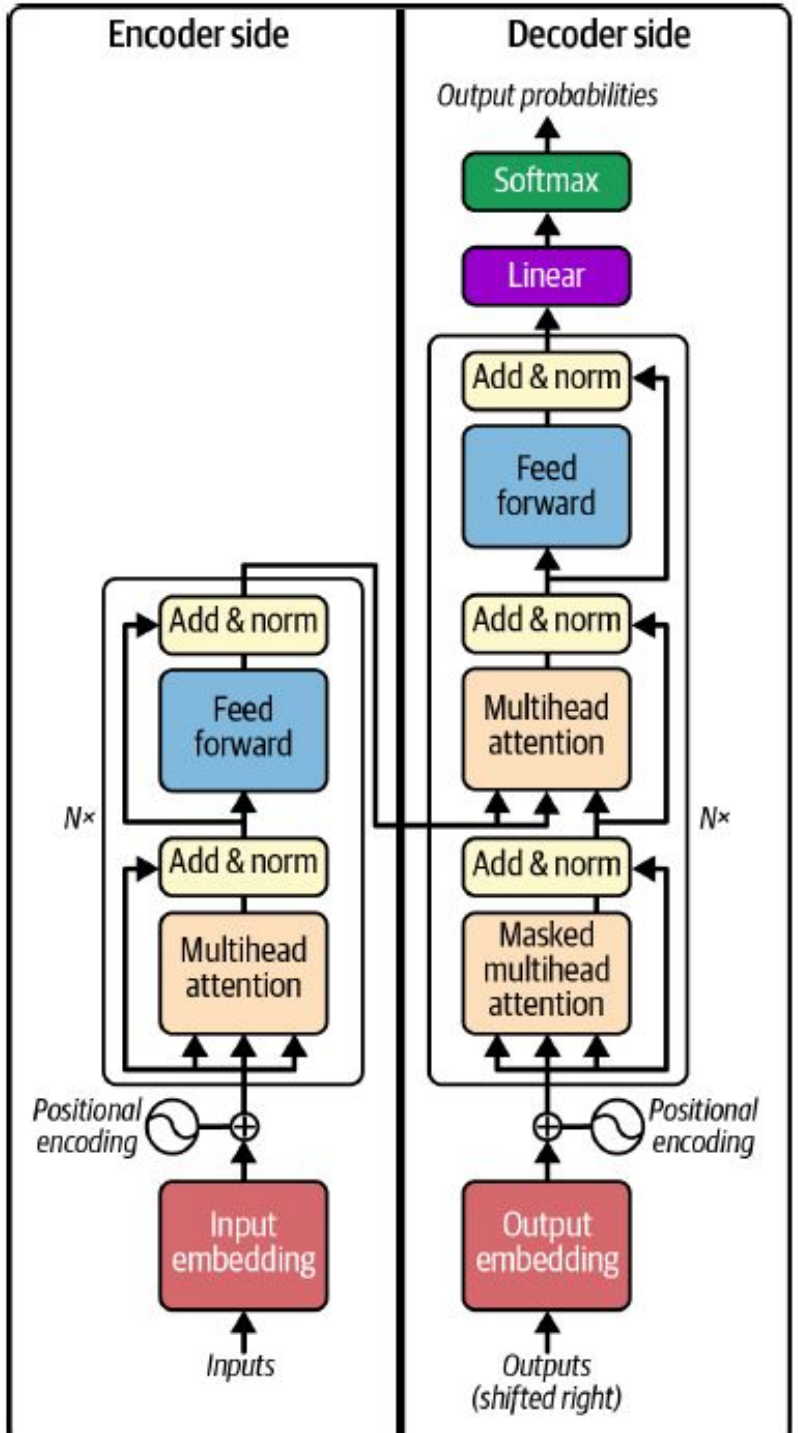
5. The output token is provided as the next input into the decoder. At this point, the process recurrently loops back and forth from step 4 to step 5.

6. Finally, the decoder emits a special “end” token, indicating that the decoding process is complete. The limited thought vector could transfer only a limited amount of information to the decoder.

- A 2015 paper, “Neural Machine Translation by Jointly Learning to Align and Translate”, introduced a new approach to addressing this bottleneck.
  - Earlier models used one single summary of the whole sentence.
  - Attention-based models store information for every word.
  - The decoder checks all the words and pays more attention to the important ones while generating the output.

- AI community, culminating in the 2017 Google Research paper “Attention Is All You Need”, which introduced the transformer architecture

Transformer architecture



Transformer Architecture

- The **Transformer** model keeps the same basic structure as the **Seq2Seq model**:
  - An **Encoder** receives the input words (tokens).
  - A **Decoder** generates the output words (tokens).
- the Transformer **does not use RNNs or recurrent connections**.
- uses **only the Attention mechanism** to understand the relationships between words.

Advantages of Transformers

- Faster to train because all words can be processed **at the same time (in parallel)**.
- Better at understanding long-range relationships between words.
- More accurate than Seq2Seq on many language tasks.

Limitation of Transformers

- A Transformer can process **only a fixed number of input and output tokens at a time** (called the **context window**).
- If the input is longer than this limit, it must be split or truncated.

GPT Models

- **GPT (Generative Pre-trained Transformer)** is built on the Transformer architecture.
- Modern GPT models have increased the context window significantly, but there is still a limit on how much text they can process in a single request.

GPT Version

Model	Release date	Parameter count	Training data	Training cost
GPT-1	June 11, 2018	117 million	BookCorpus: 4.5 GB of text from 7,000 unpublished books of various genres	1.7e19 FLOP
GPT-2	February 14, 2019 (initial); November 5, 2019 (full)	1.5 billion	WebText: 40 GB of text and 8 million documents from 45 million web pages upvoted on Reddit	1.5e21 FLOP
GPT-3	May 28, 2020	175 billion	499 billion tokens consisting of Common Crawl (570 GB), WebText, English Wikipedia, and two books corpora (Books1 and Books2)	3.1e23 FLOP
GPT-3.5	March 15, 2022	175 billion	Undisclosed	Undisclosed
GPT-4	March 14, 2023	1.8 trillion (rumored)	Rumored to be 13 trillion tokens	Estimated to be 2.1e25 FLOP

- When discussing **training cost**, FLOPs measure **how much computation is required to train an AI model**.
- Training cost (FLOPs) = Total number of mathematical calculations performed while training a model.

More FLOPs mean:

- More powerful hardware (GPUs/TPUs) is needed.
- Longer training time.
- Higher electricity and cloud computing costs.

Prompt Engineering

- The input into the model is called the *prompt*—it is a document, or block of text.
- *Prompt engineering*, is the practice of crafting the prompt so that its completion contains the information required to address the problem at hand.
- LLM-based application - prompt construction and the interpretation of the answer are done programmatically.
- To build a quality piece of software and a quality UX, the prompt engineer must create a pattern for iterative communication among the user, the application, and the LLM.

- The user –problem- the application, the application - pseudodocument to be sent - LLM, the LLM completes the document, and finally, the application parses the completion and back - user or otherwise performs an action on the user’s behalf.
- PE - communication is structured in a way that best translates among very different domains, the user’s problem space, and the document space of LLMs

PE levels of sophistication

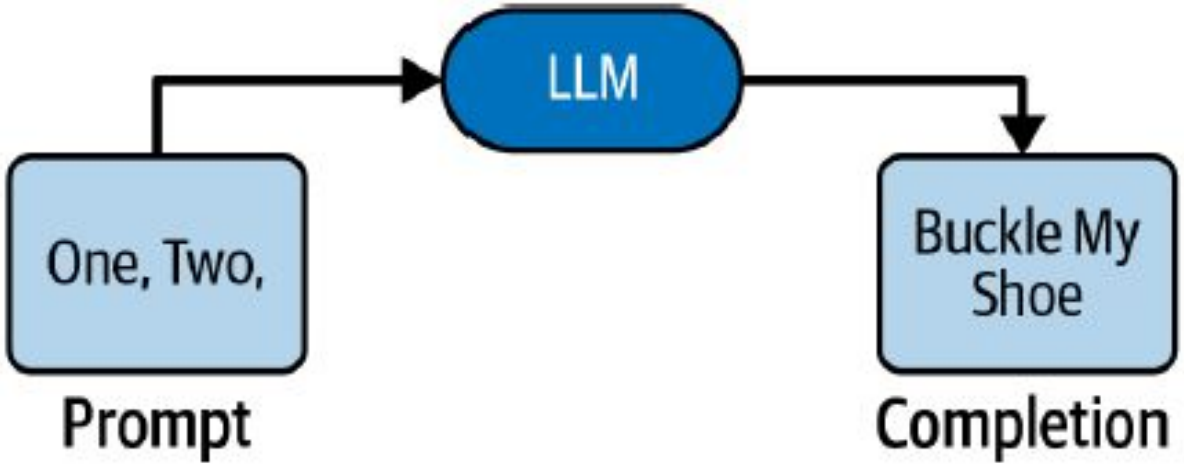
- Basic - thin application layer
  - ChatGPT- special ChatML markdown
  - GitHub Copilot
- Next level involves modifying and augmenting the user's input into the model
  - user's speech to text
- When the interactions with the LLM become *stateful*, meaning they maintain context and information from prior interactions
  - chat application

- LLM-based application tools that allow the LLM to reach out into the real world by making API requests to read information or to create or modify assets.
  - LLM-based email application
- how to provide the LLM application with agency. the ability to make its own decisions
  - AutoGPT

Understanding LLM

What Are LLMs?

- An *LLM* is a service that takes a string and returns a string: text in, text out. The input is called the *prompt*, and the output is called the *completion* or, the *response.*



- What happens untrained LLM?
- Training takes skill, compute, and time so most LLM applications use off-the-shelf generalist models (known as *foundation models*) that are already trained (maybe after a bit of fine-tuning).
- LLMs are trained using a large set of documents (strings) known as the *training set*.
- Training Set - books, articles, conversations on platforms such as Reddit, and code on sites such as GitHub.

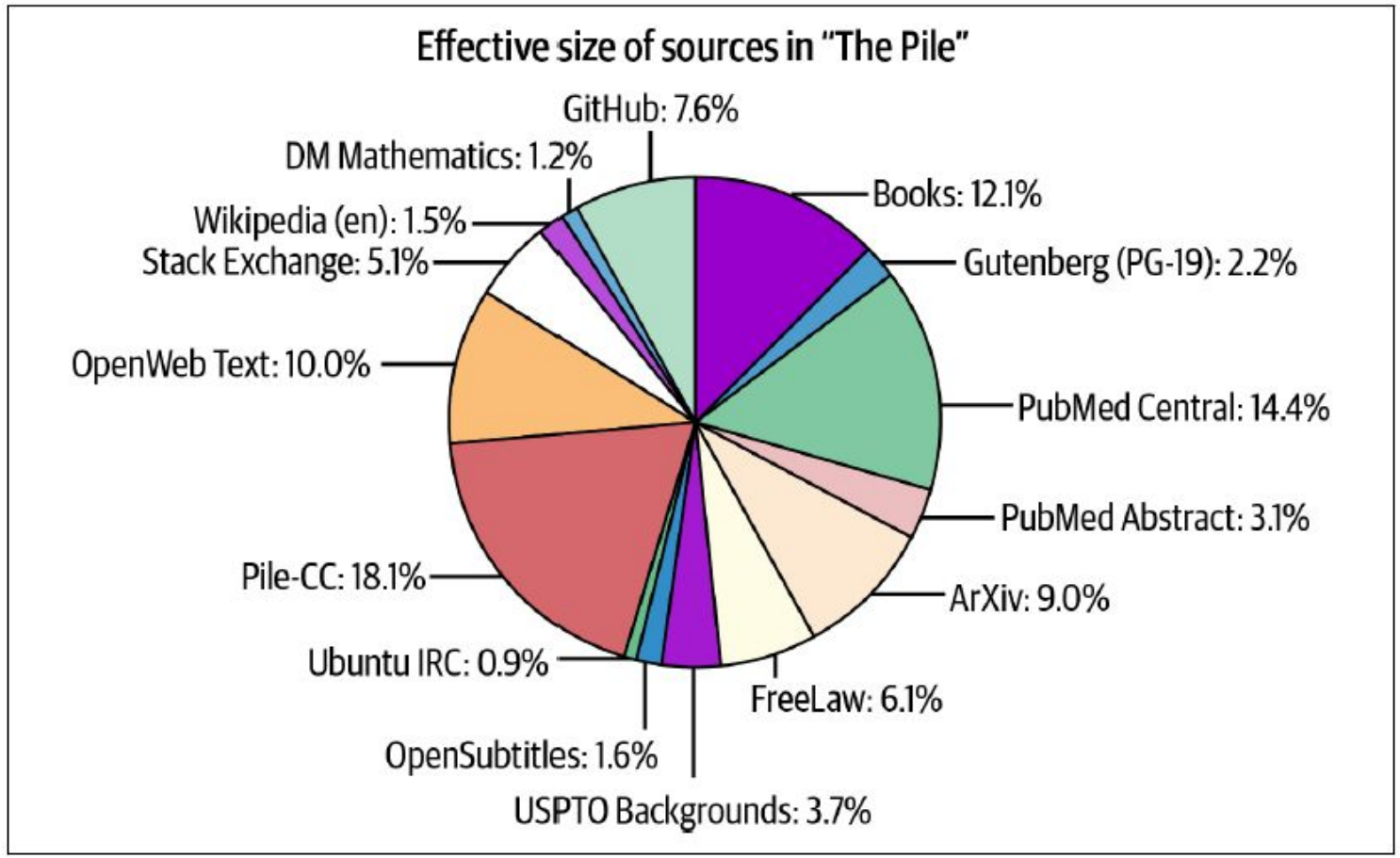


Figure 2-2. Composition of "The Pile", a popular open source training set comprising a mixture of factual prose, fictional prose, dialogues, and other internet content

- inner architecture of the LLM and the training procedure must prevent LLM to recite the training set by heart but to apply the patterns.
- That prevention sometimes fails, and instead of learning facts and patterns, the model learns chunks of text by rote—*overfitting*.
- Large-scale overfitting should be rare in off-the-shelf models.



Assume you have picked a document from the training set at random. All you know about it is, it starts with the prompt. What is the statistically most likely continuation? That's the LLM output you should expect.

Completing a Document

- *Yesterday, my TV stopped working. Now, I can't turn it on at*
- For a text that starts like this, what might be the statistically most likely completion?
- 1. y2ior3w
- 2. Thursday.
- 3. all.
- Let's take completion 3 as given and run the LLM a bit further:
- *Yesterday, my TV stopped working. Now, I can't turn it on at all.*

- a. This is why I chose to settle down with a book tonight.
- b. Shall we watch the game at your place instead?
- c. \n
- \n
- First, try unplugging the TV from the wall and plugging it back in.

Human Thought Versus LLM Processing

- write a blog post about a podcast
- You might start writing the following: In their newest installment of `The rest is history`, they talk about the Hundred Years' War (listen on acast at http://.
- do a quick internet search. Hopefully, you find the correct link: shows.acast.com/the-rest-is-history-podcast/episodes/321-hundred-years-war-a-storm-of-swords. Or maybe you can't find it.
- The model can't google or edit, so it just guesses

- OpenAI’s text-curie-001, a small variant of GPT3, and this LLM completed the URL as follows:
- http://www.acast.com/the-rest-is-history-episode-5-the-Hundred-Years-War- \1411-1453-with-dr-martin-kemp)

Hallucinations

- The fact that LLMs are trained as “training data mimic machines” has unfortunate consequences:
  - *Hallucinations* - factually wrong but plausible-looking pieces of information produced confidently by the model. Common problem when using LLMs, either ad hoc or within applications.
- If prompt references something that doesn’t exist, an LLM will typically continue to assume its existence.
- Documents that start out with wrong claims and then correct themselves halfway through are rare.
- So the model will typically assume its prompt to be true, and this is known as *truth bias*.

How LLMs See the World

LLMs Don’t Read Characters Directly

- LLMs consume and produce **strings (text)**.
- Unlike humans, they don’t naturally process text character-by-character.
- Character-level reasoning requires significant effort.
- Even advanced LLMs can struggle with simple letter-counting questions.

How Humans Read Text

- Humans quickly group letters into **words**.
- Our brain automatically corrects many typos while reading.
- We can often understand text even when letters within words are scrambled.
- Maintaining **word boundaries** makes garbled text easier to understand.

How LLMs Process Text

- LLMs also do not directly read individual letters.
- Text is first divided into smaller units called **tokens**.
- Tokens are usually **3–4 characters long**.
- Common words or letter sequences can form longer tokens.

What Are Tokens?

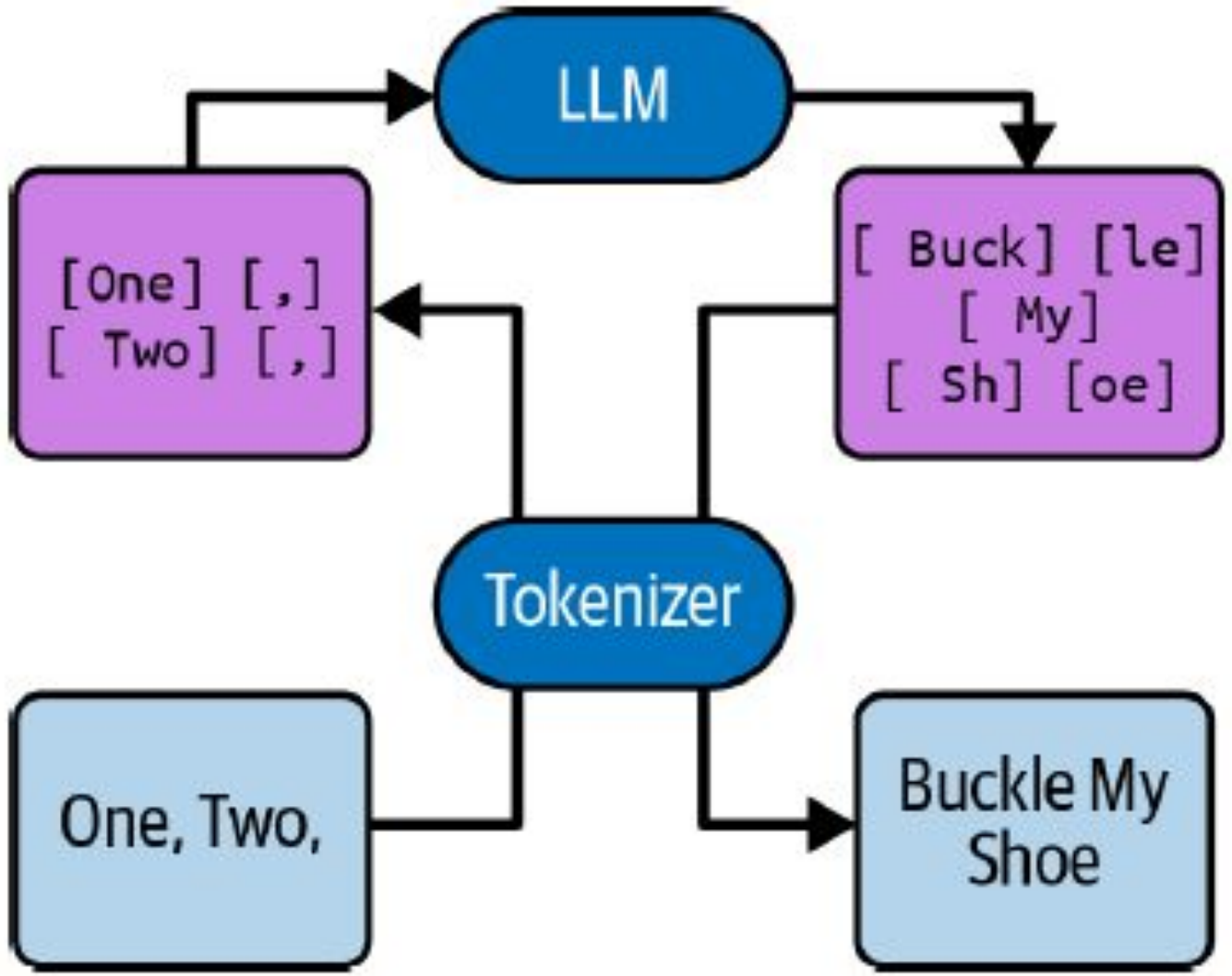
- A **token** is a chunk of text processed by an LLM.
- Examples can include:
  - Parts of words
  - Complete common words
  - Punctuation
  - Spaces or whitespace-connected text
- The complete collection of tokens is called the **vocabulary**.

Tokenization Process

- Text → **Tokenizer** → Tokens → **LLM**
- The tokenizer converts text into a sequence of tokens.
- Tokens are represented internally as **numbers**.
- The LLM processes these numerical representations.

From LLM Output Back to Text

- The LLM generates a sequence of tokens.
- Generated tokens are converted back into text.
- Process:
  Input Text → Tokenizer → Numerical Tokens → LLM → Output Tokens → Text


LLM
[One] [,]
[Two] [,]
[Buck] [le]
[My]
[Sh] [oe]
Tokenizer
One, Two,
Buckle My
Shoe

Difference 1: LLMs Use Deterministic Tokenizers

- **Humans vs. LLMs**
- Humans interpret letters **fuzzily**.
- We usually recognize a word even when it contains a typo.
- Our brain looks for the word that is **most similar** to what we see.
- Example: We can often understand **“gohst”** as “ghost”.

How LLMs Handle Typos

- LLMs use **deterministic tokenizers**.
- A tokenizer converts text into predefined tokens.
- Correct and misspelled words can produce **different token sequences**.
- This makes typos more noticeable to the model.
- **Example – “ghost” vs. “gohst”**
- **“ghost”** → represented as a single token in the GPT tokenizer.
- **“gohst”** → split into multiple tokens: **g – oh – st**.
- The different tokenization makes the typo distinguishable.

Why LLMs Still Handle Typos Well

- LLMs encounter many **typos and misspellings during training**.
- Their training data contains naturally occurring mistakes.
- As a result, LLMs can often understand the intended meaning despite typos.
- **Key idea:** Tokenization exposes the difference, but training helps the model interpret it correctly.

Difference 2: LLMs Can’t Slow Down and Examine Letters

Humans vs. LLMs

- Humans can consciously examine **individual letters** when needed.
- LLMs rely on their **built-in tokenizer**.
- LLMs cannot simply “slow down” to inspect characters one by one.
- This creates difficulties with tasks involving **breaking or rebuilding tokens**.

Why Letter Manipulation Is Difficult

- LLMs may know which letters make up a token.
- However, **splitting tokens and reassembling them** is difficult.
- Tasks such as reversing letters can therefore produce unexpected errors.
- Even simple-looking character manipulations may be challenging for LLMs.

Example – Reversing Words

- Reversing a word seems like a simple pattern-manipulation task.
- But the LLM must first **break tokens apart**.
- It then needs to **reassemble the pieces in reverse order**.
- Errors can occur in both the reversal and the **re-reversal**.

Key Lesson for Application Builders

- Avoid tasks that require precise **subtoken/character-level manipulation**.
- If possible, move such logic **outside the LLM**.
- Use traditional programming or syntactic rules for exact character operations.

- **Example – Scattergories**
- Games like Scattergories require exact syntactic conditions.
- Examples:
  - Activist starting with **W**
  - European country starting with **“Sw”**
  - Fruit containing **3 occurrences of “R”**
- Asking the LLM to perform all these checks can lead to errors.

- **Better Approach – LLM + Syntactic Logic**
- Use the LLM to generate a **large list of possible answers**.
- Use normal programming/syntactic logic to **filter and verify** them.
- Example:
  LLM → Generate countries → Code checks “Sw” → Valid answers
- This divides the task according to each tool's strengths.

Difference 3: LLMs See Text Differently

- **Humans vs. LLMs – Visual Understanding**
- Humans naturally recognize **visual properties of letters**.
- We can easily distinguish:
  - Round vs. square letters
  - Different letter shapes
  - ASCII art patterns
- LLMs do not inherently “see” these properties in the same intuitive way.
- They may rely on patterns learned from training data.

- **Accented Letters**
- Humans usually treat accented letters as **variants of the same letter**.
- Example: a, á, à, ä are easily recognized as related.
- Humans can often ignore accents while reading.
- For an LLM, handling these variations can require **additional processing**.

The Capitalization Problem

- Capitalization looks simple to humans.
- For LLMs, uppercase and lowercase forms can correspond to **different tokens**.
- Example:
  - a and A may occur in different token representations.
- This can make simple capitalization tasks surprisingly difficult.

Tokenization Changes with Capitalization

- The same phrase can be tokenized differently depending on capitalization.
- Example:
  - strange new worlds → [str] [ange] [new] [worlds]
  - STRANGE NEW WORLDS → [STR] [ANGE] [NEW] [WOR] [L] [DS]
- More tokens mean more complexity for the model.

- **Another Example – “gone”**
- gone → **one token**
- GONE → **two tokens: [G] [ONE]**
- The relationship between uppercase and lowercase forms is therefore **not always one-to-one**.

Why This Matters

- LLMs have learned capitalization patterns from training data.
- Better models handle capitalization more effectively.
- However, processing these token differences still consumes **model capacity**.
- That capacity could otherwise be used for the main task.

Lesson for Prompt Engineers

- Avoid asking an LLM to perform unnecessary **capitalization or formatting tasks**.
- Use conventional programming tools for simple text transformations.
- Keep the LLM focused on tasks requiring **reasoning, language understanding, and generation**.

Counting Tokens

- **Every Model Has a Fixed Tokenizer**
- You **cannot mix and match** tokenizers and models.
- Each LLM is designed to work with a specific tokenizer.
- Understanding your model's tokenizer is important when building LLM applications.
- Tools such as **Hugging Face** and **tiktoken** can be used to inspect tokens.

- **Why Count Tokens?**
- The most common use of a tokenizer is simply **counting tokens**.
- Token count determines how long text is from the model's perspective.
- More tokens generally mean:
  - More processing time
  - More computational cost
  - Potentially higher API cost

- **Tokens and Processing Cost**
- Prompt processing time scales roughly with the **number of input tokens**.
- Generating output takes time based on the **number of output tokens**.
- Computational requirements also increase with token count.
- Many LLM services therefore **charge based on tokens processed or generated**.

- **Tokens and Context Window**
- A model's **context window** is the maximum amount of text it can handle at one time.
- Context size is measured in **tokens**, not characters or pages.
- The total of:
  Prompt tokens + Completion tokens ≤ Context Window
- Context windows can represent many pages of text.

- **Why Token Counting Matters**
- Large context windows can be tempting to fill completely.
- Overloading the context window can cause problems.
- Token counting helps developers:
  - Stay within context limits
  - Control prompt size
  - Manage costs
  - Improve application performance

- **Characters ≠ Tokens**
- There is **no fixed conversion** between characters and tokens.
- Token count depends on:
  - The text
  - The language
  - The tokenizer
- For English text, a common GPT tokenizer averages roughly **4 characters per token**.

- **Token Efficiency Varies**
- Tokenizers are generally optimized for **English**.
- Other languages may require **more tokens** for the same amount of text.
- Random numbers are less efficient: **a little over 2 characters/token**.
- Random alphanumeric strings, such as cryptographic keys, can have **fewer than 2 characters/token**.
- Rare Unicode characters can require multiple tokens.

One Token at a Time

Auto-Regressive Models

- **How an LLM Generates Text**
- An LLM predicts the **next token** based on the existing prompt.
- The predicted token is added to the prompt.
- The model then predicts the **next token** again.
- This process continues until the response is complete.

- **What Is Autoregressive Generation?**
- **Autoregressive** means predicting one token at a time.
- Each new prediction depends on the **previous tokens**.
- Process:
  Prompt → Predict Token → Add Token → Predict Next Token → Repeat

- **Phone Keyboard Analogy**
- Smartphone keyboards often provide **three-word suggestions**.
- Running an LLM is similar to repeatedly selecting one suggested word/token.
- The model continuously chooses what is statistically most likely to come next.

- **LLMs Cannot Pause and Reflect**
- Humans can stop while writing to:
  - Think
  - Check their work
  - Reflect
  - Change their approach
- An LLM generates **one token at every step**.
- It does not automatically get extra time to “think longer.”

- **No Easy Backtracking**
- Once an LLM outputs a token, it is **committed to that output**.
- It cannot simply erase a previously generated token.
- It may continue following an incorrect direction.
- This can make an LLM appear **stubborn or inconsistent**.

- **Why Mistakes Can Continue**
- The model generates text sequentially.
- Earlier mistakes become part of the **new context**.
- Future predictions are influenced by those mistakes.
- The model may continue down a path that no longer makes sense.

- **Role of the Application Designer**
- LLMs do not inherently provide reliable **mistake recognition and backtracking**.
- Applications may need to add mechanisms for:
  - Checking outputs
  - Detecting errors
  - Re-running the model
  - Correcting or revising responses
- **Key idea:** Build error detection and correction into the application when needed.

Temperature HyperParameter and Probabilities

- LLM, it turns out that actually, it computes the probability of *all possible tokens* before choosing a single one. The process under the hood that chooses the actual token is called ***sampling***.
- LLM doesn't just compute the most likely token; it computes the likelihood of all the tokens

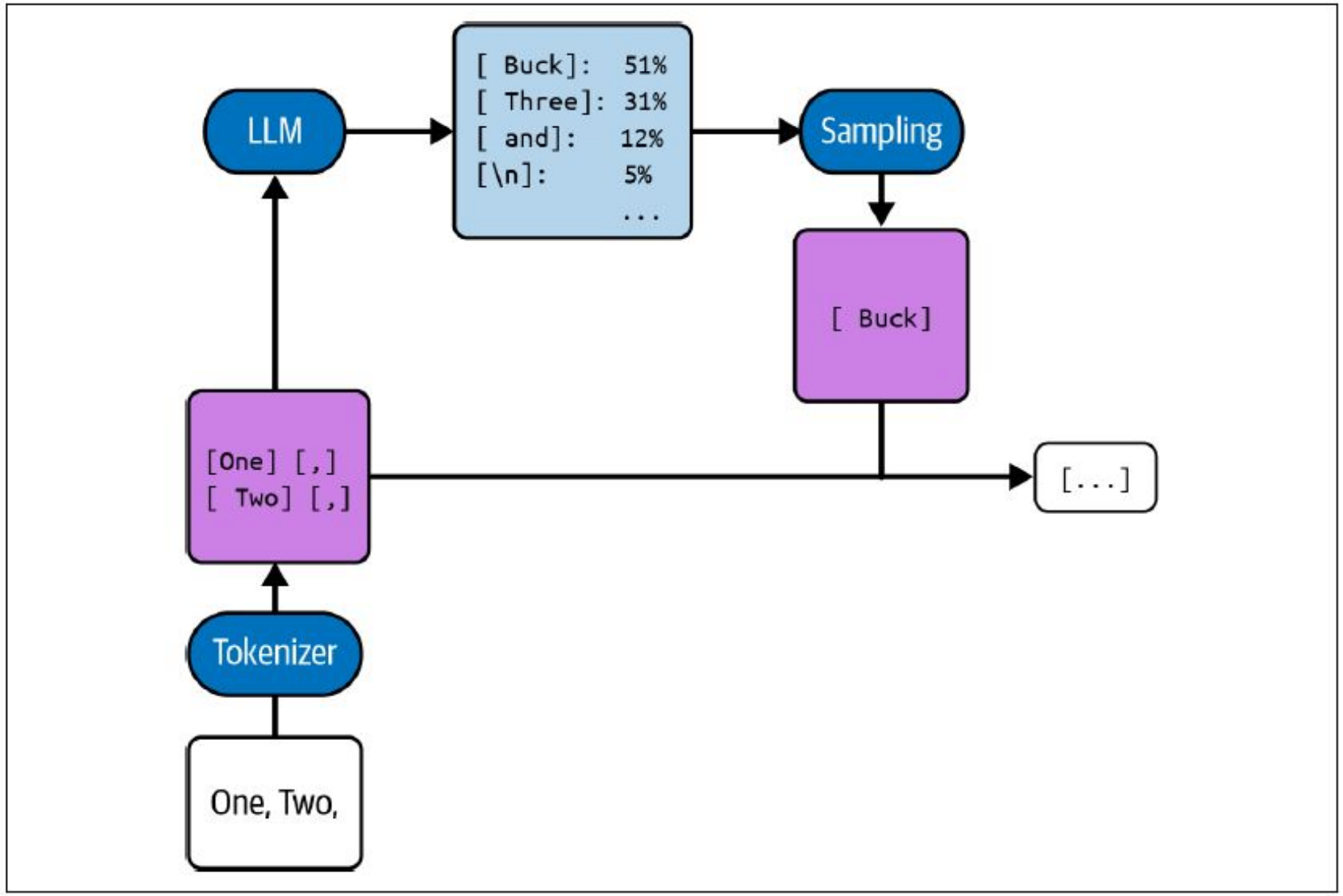


Figure 2-11. The sampling process in action

- The model typically returns them as *logprobs* (i.e., the natural logarithms of the token's probability).
- The higher the logprob, the more likely the model considers this token to be.
- Logprobs are never bigger than 0 because a logprob of 0 would mean that the model is certain that this is the next token.
- Expect the most likely token to have a logprob between –2 and 0

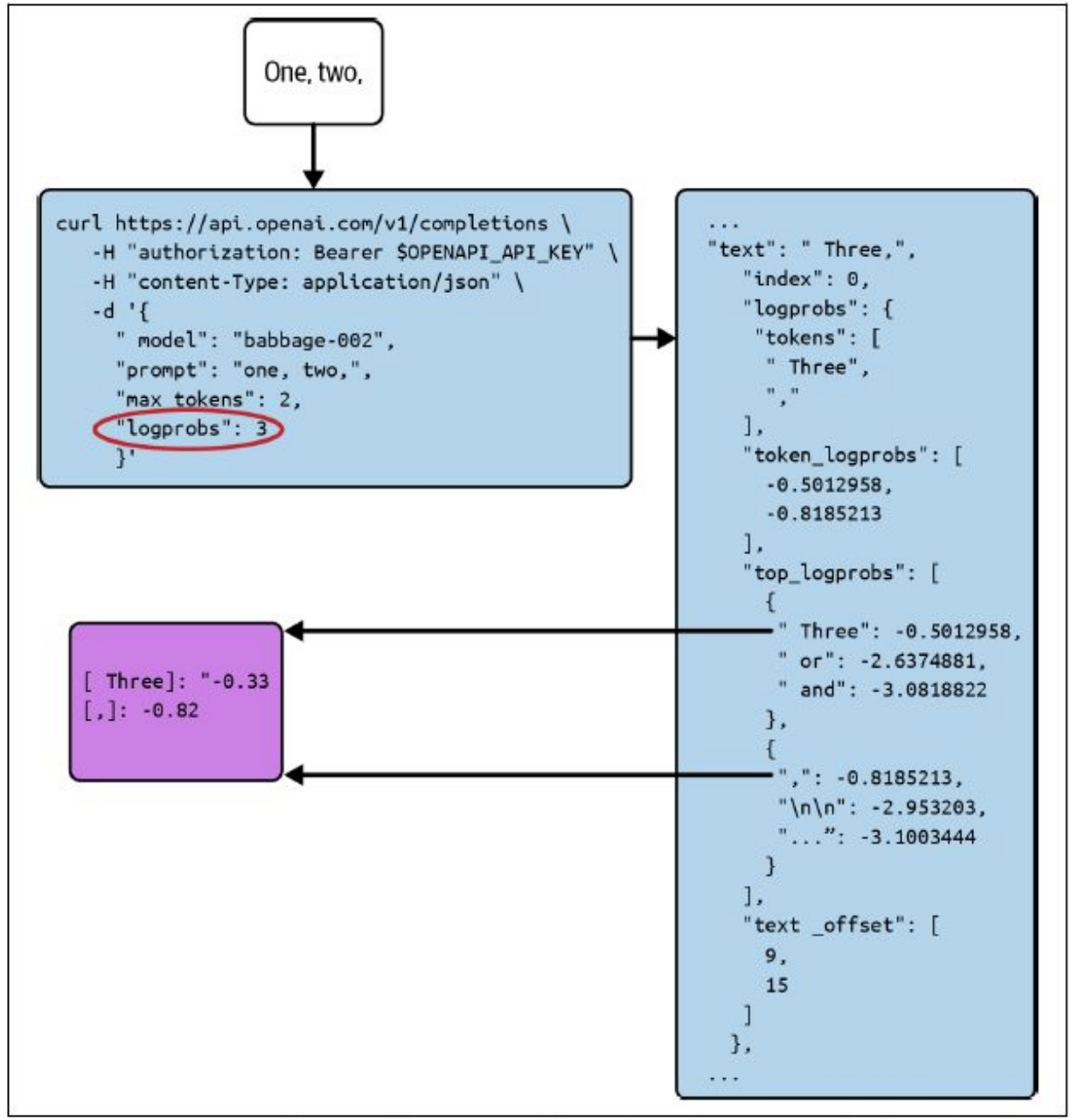


Figure 2-12. An example API call requesting logprobs and extracting the logprobs of the chosen completion

- logprobs to 3 means that the logprobs for the three most likely tokens will be returned.
- if you have a way of automatically testing your completions, you may want to generate a couple of alternatives and throw out the bad ones.
- The typical way to do this is by using a *temperature* greater than 0.
- The temperature is a number of at least zero that determines how "creative" the model should be.

$$p(\text{token}_i) = \frac{\exp\left(\text{logprob}_i / t\right)}{\Sigma_j \exp\left(\text{logprob}_j / t\right)}$$

- *0* - You want the most likely token. No alternatives. This is the recommended setting when correctness is paramount.
- *0.1–0.4* - If there's an alternative token that's only slightly less likely than the front-runner, you want some small chance for that to be picked.
- *0.5–0.7* - You want a greater impact of chance on the solution, and you are fine with getting completions that are "inaccurate" in the sense that sometimes, a token will be chosen even though the model thinks another alternative is clearly more likely.
- *1* - You want the token distribution to mirror the statistical training set distribution.

- *> 1* - You want a text that's "more random" than the training set. This means the model is less likely to pick the "standard" continuation than the typical document

The Transformer Architecture

- *transformer*, which is the architecture used by all modern LLMs
- Each minibrain starts out by being told which token it's sitting on and its position in the document.
- The minibrain keeps thinking about this for a fixed number of steps, known as *layers*. During this time, it can receive information from the minibrains to the left.
- The minibrain's task is to understand the document from the perspective of its location, and it uses this understanding in two ways:

1. In all steps before the last one, it shares some of its intermediate results with the minibrains to its right.
2. For the last step, it's asked to make a prediction of what the token immediately to its right would be.

- The minibrains are clones of each other: their processing logic is the same, and all that differs is the inputs: which token they start with and which intermediate results they get told of by the minibrains to their left.

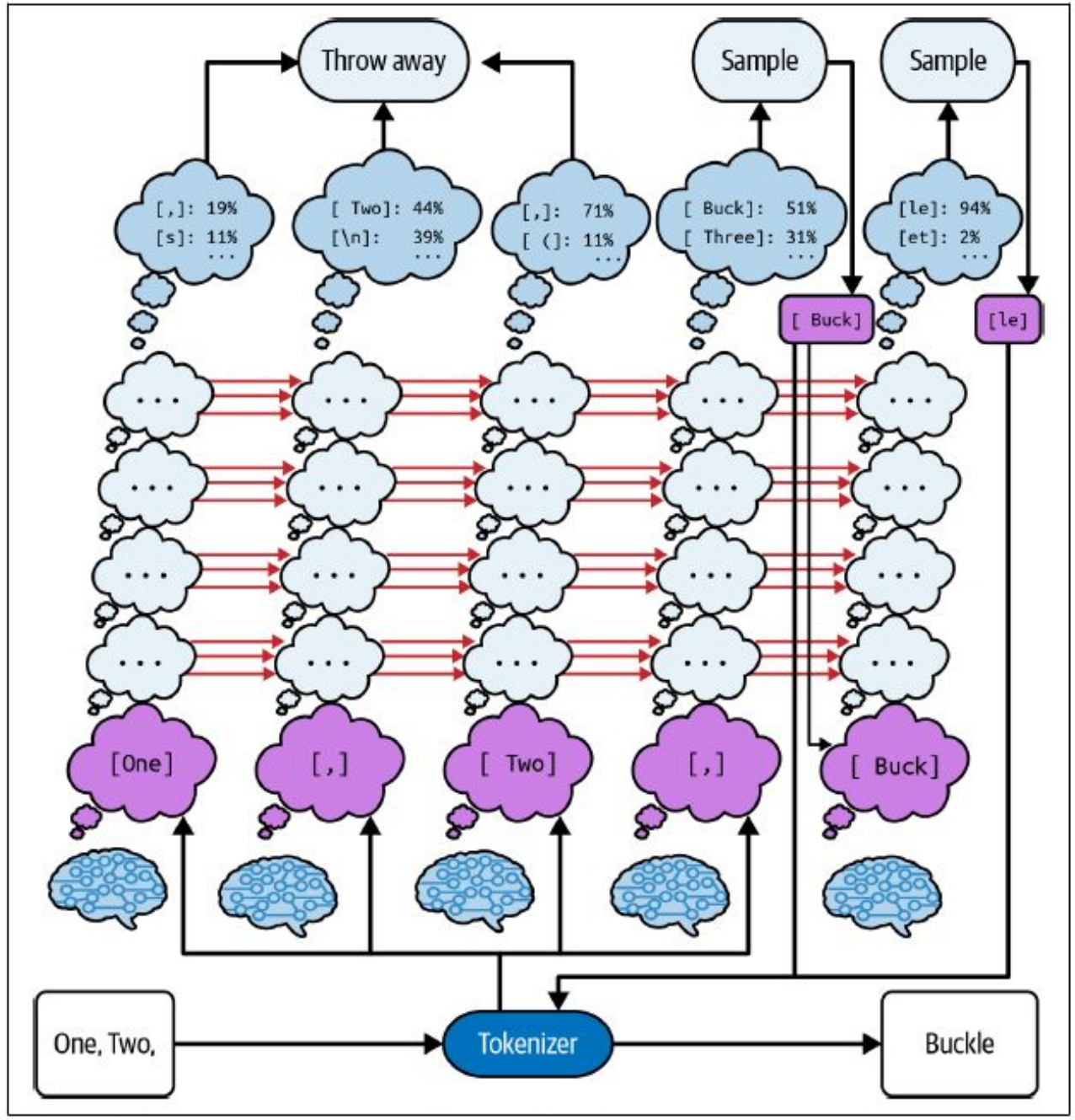


Figure 2-14. The inner workings of the model producing one token—later layers are drawn on top of previous layers

Working

- 1. Each minibrain has some things it wants to know, so it submits a couple of questions, in the hope they might get answered by another minibrain. Let's say that one minibrain sits upon the token [my]. The minibrain would like to know who that might refer to, so a reasonable question would be to ask, "Who is talking?"
- 2. Each minibrain has some things it can share, so it submits a couple of items, in the hope they might be useful to another minibrain. Let's say one minibrain sits upon the token [Susan], and it's already learned before that this token is the last word of an introduction, like "Hello, I'm Susan." So in case it might help another minibrain down the line, it will submit the information, "The person talking right now is Susan."
- 3. Now, every question is matched up with its best-fitting answer. "Who is talking?" matches up very well with "The person talking right now is Susan."
- 4. The best-fitting answer to each question is revealed to the minibrain that asked the question, so the minibrain at the token [my] gets told "The person talking right now is Susan." Of course, while the minibrains from this example talk to each other in English, in reality, they use a "language" that consists of long vectors of numbers13 and that is unique to every LLM, since it's something the LLM "invents" during training.

- In modern LLMs, this Q&A mechanism obeys one more constraint, which is called *masking*: not *all* minibrains can answer a question; only the ones to the *left* of the minibrain asking the question can answer it.

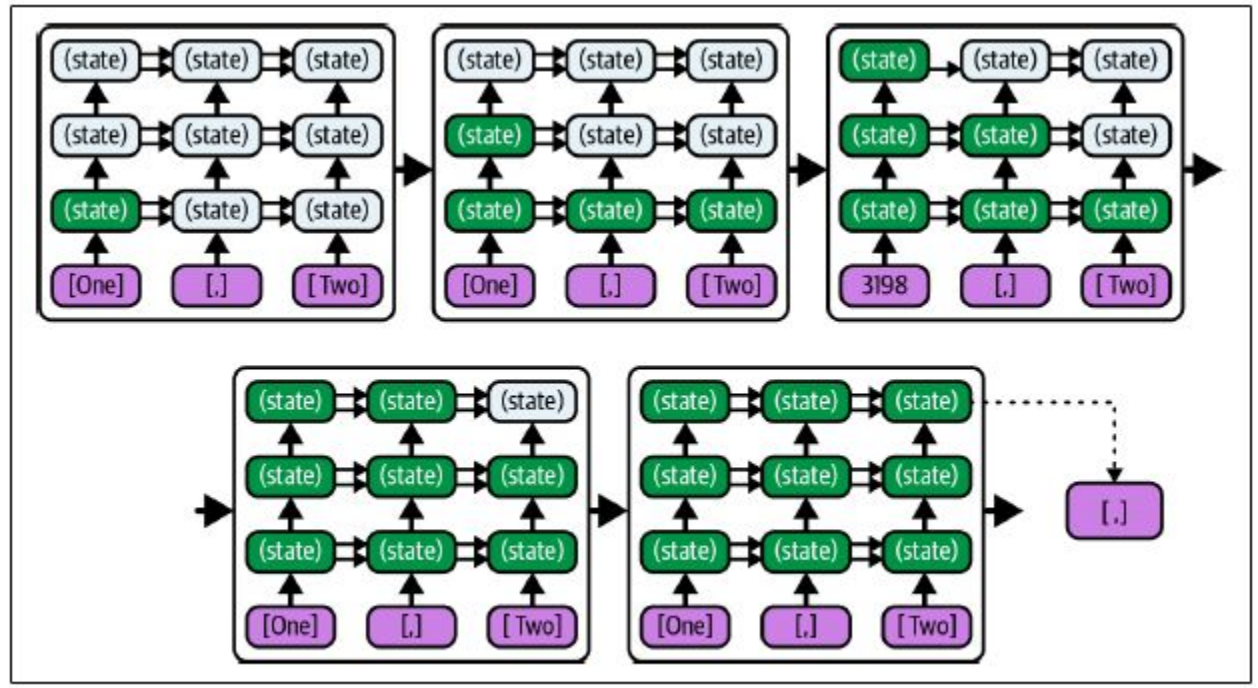


Figure 2-15. Calculating the inner state of an LLM

- first (at the upper left), only the lowest layer at the first token can be computed. Next (at the upper middle), both the second-lowest layer at the first token and the lowest layer at the second token can be computed.
- One step later (at the upper right), the third layer can be computed at the first token, the second layer at the second token, and the first layer at the third token…all the way until all states are computed and a new token can be sampled.

- This triangle structure reflects a general “backward-and-downward” direction.